FUNDAMENTAL MACHINE LEARNING USING PYTHON FOR PUBLIC HEALTH

DAY 2 / 2

Fundamental Machine Learning using Python for Public Health

Logistic Regression, Neural Networks & Deep Learning

Teerapong Panboonyuen, Ph.D. (Kao)

College of Computing, Khon Kaen University

https://github.com/kaopanboonyuen/KKU_Biostat_ML_2026s2

WHERE WE LEFT OFF

Recap Day 1 & Today's Roadmap

1 Yesterday

Python refresher → pandas EDA → feature engineering → Decision Tree & Random Forest → full evaluation suite.

2 A · Logistic Regression

The single most important algorithm in AI history, explained from first principles: sigmoid, loss, gradient descent.

3 B · Neurons → Networks

See how one neuron IS logistic regression, then build real PyTorch neural networks — MLPs and a 1D CNN.

4 C · Model Comparison

Six models, one test set, one metric suite — a fair, reproducible benchmark.

5 D · Research & Ethics

A guide to doing AI/ML research the right way, and using AI responsibly in public health.

6 Goal

Leave today able to read any deep-learning paper's Methods section and reproduce it.

BY THE END OF DAY 2

Learning Objectives

- Understand Logistic Regression in full mathematical detail — the model that started it all.
- See exactly how a single artificial neuron (a sigmoid unit) IS a logistic regression.
- Build and train PyTorch neural networks, from a simple MLP to a 1D CNN.
- Read a research paper's Methods section and translate hyperparameters into working code.
- Handle imbalanced clinical data properly using class weighting.
- Compare Decision Tree / Random Forest / Logistic Regression / Neural Networks on the same data & metrics.
- Close with a recap, an AI/ML research roadmap, and essential ethics guidance for public health.

SETUP

Reloading the Data & Feature Scaling

1 Same Pipeline

We repeat Day 1's exact imputation, binning, encoding, and composite features — same 37 model-ready columns.

2 Why Scale Now?

Trees split on raw thresholds, so scale never mattered. But Logistic Regression & Neural Nets compute weighted sums — unscaled features break the optimizer.

3 StandardScaler

Rescales every feature to mean 0, standard deviation 1.

4 Golden Rule

Fit the scaler ONLY on training data, then transform the test set — fitting on test data leaks information and inflates results.

A

FIRST PRINCIPLES

Logistic Regression, Explained From the Ground Up

Arguably the single most important algorithm in AI history — understanding it fully unlocks Deep Learning.

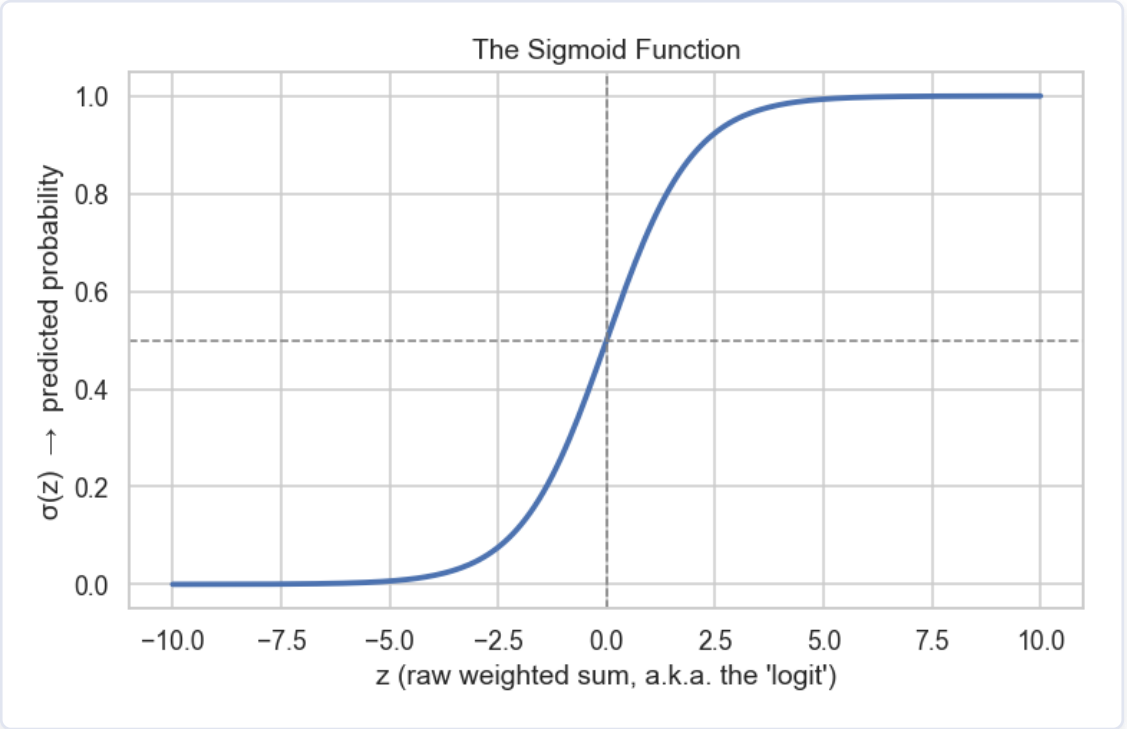
A.1 — The Problem With Linear Regression for Classification

- A plain linear model predicts $\hat{y} = w \cdot x + b$ — any real number, e.g. −3.2 or +57.
- But we want a probability of stroke, which must lie strictly between 0 and 1.
- We need a function that "squashes" any real number into the (0, 1) range — enter the sigmoid function.

A.2 — The Sigmoid Function

σ(z) = 1 / (1 + e^−z)

- As z → +∞, σ(z) → 1 (very confident "stroke").
- As z → −∞, σ(z) → 0 (very confident "no stroke").
- At z = 0, σ(z) = 0.5 — the default decision boundary.
- This single S-shaped curve is the building block of every neuron in a neural network.



A.3 — The Full Logistic Regression Model

Step 1 — Logit	$z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$	A weighted linear combination of the patient's features, exactly like linear regression.
Step 2 — Probability	$\hat{p} = P(\text{stroke} = 1 \mid x) = \sigma(z)$	Squash the logit with sigmoid to get a valid probability between 0 and 1.
Step 3 — Class	$\hat{y} = 1 \text{ if } \hat{p} \geq 0.5, \text{ else } 0$	Threshold the probability to obtain the final 0/1 prediction (threshold is tunable!).

A.4 — How the Model Learns: Binary Cross-Entropy

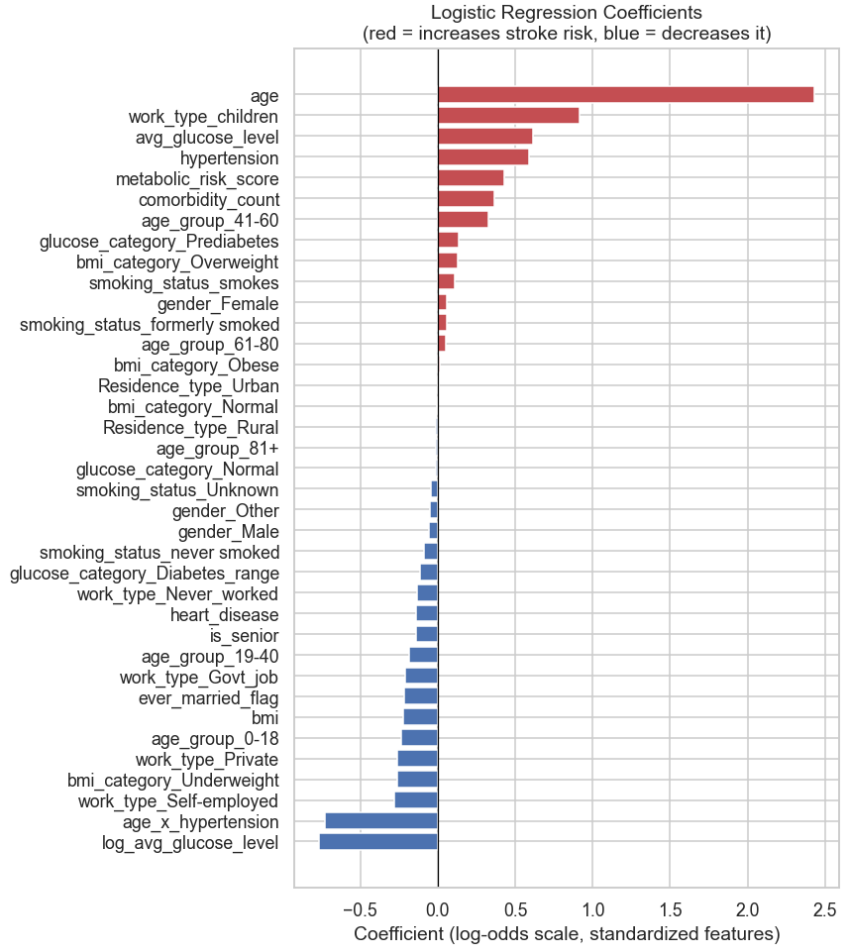
We need to measure "how wrong" a predicted probability $\hat{p}$ is, given the true label $y \in \{0, 1\}$.

Log Loss	$L(y, \hat{p}) = -[y \cdot \log(\hat{p}) + (1-y) \cdot \log(1-\hat{p})]$	If y=1 and $\hat{p} \approx 1$ → loss≈0 (great). If y=1 but $\hat{p} \approx 0$ → loss→∞ (heavily penalized).
Training	$\min (1/N) \Sigma L(y_i, \hat{p}_i)$	Find weights w and bias b that minimize the AVERAGE loss across all patients.
Gradient Descent	$w \leftarrow w - \alpha \cdot \partial L / \partial w$	At every step, nudge each weight in the direction that reduces the loss. α = learning rate.

A.5 — Training with scikit-learn: Key Hyperparameters

Parameter	Controls	Tuning Intuition
C	Inverse regularization strength	Smaller C = stronger regularization = simpler, safer model.
penalty	Type of regularization (l2/l1)	l2 shrinks all weights smoothly; l1 can zero out irrelevant features.
class_weight	Rebalances the rare class	Set "balanced" — essential, stroke is rare!
max_iter	Max gradient-descent steps	Increase if you see a "did not converge" warning.
solver	Optimization algorithm	"lbfgs" (default) works well for small/medium datasets like ours.

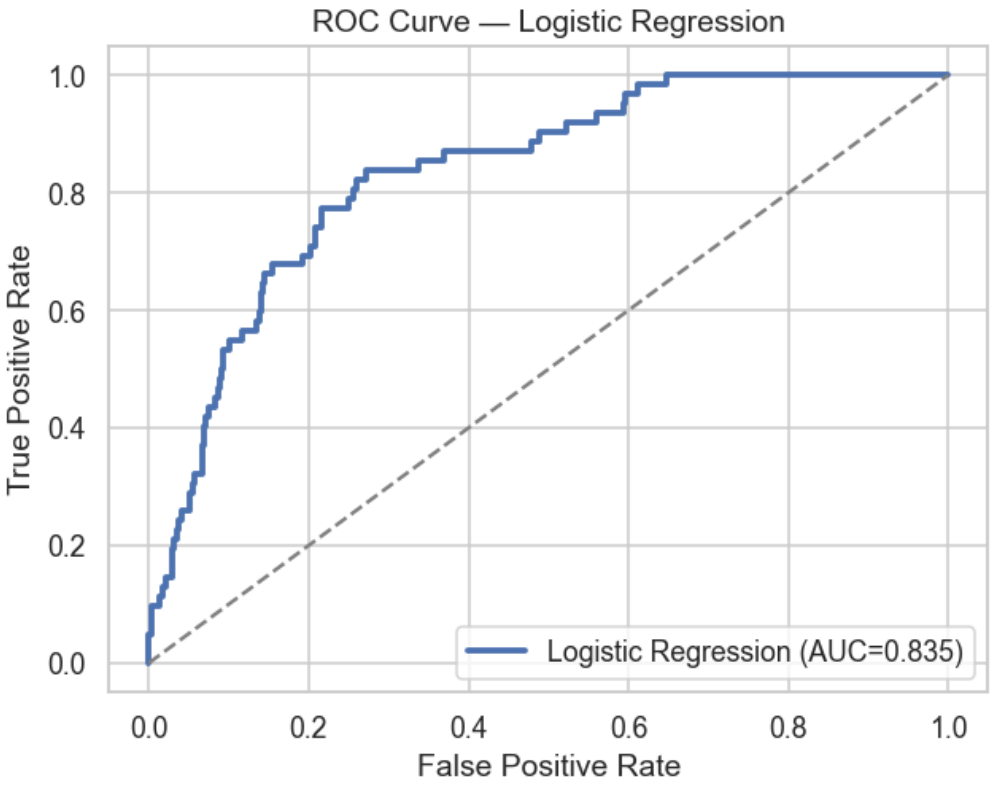
A.6 — Interpreting the Coefficients

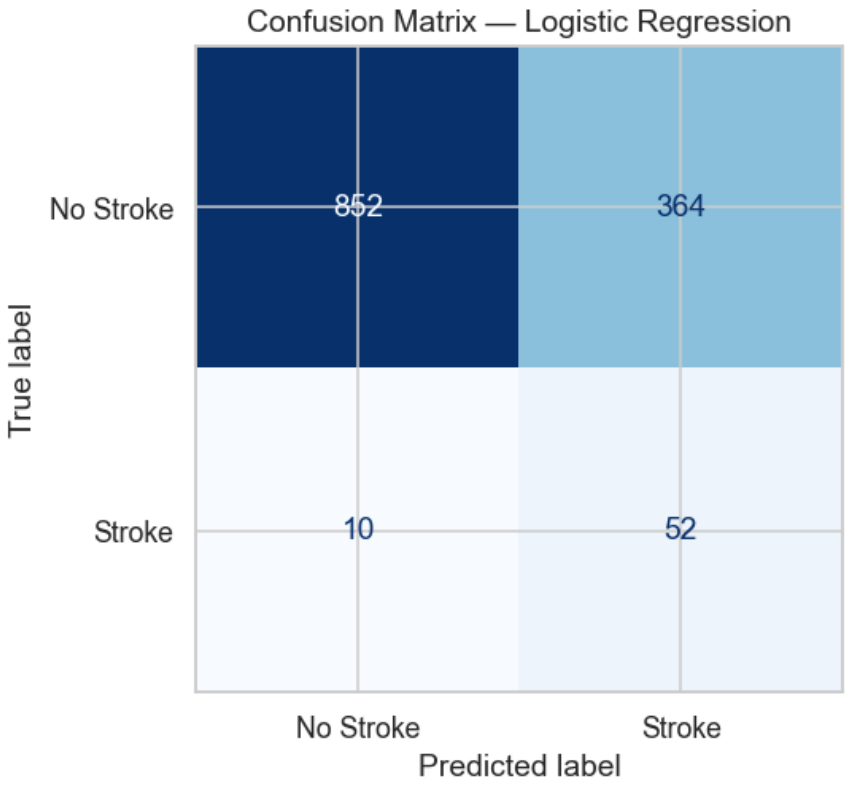


A directly interpretable equation

- Each coefficient w_i: holding everything else constant, how does a 1-SD increase in this feature change the log-odds of stroke?
- Positive (red) → increases stroke risk. Negative (blue) → decreases it.
- age, age_x_hypertension, hypertension and heart_disease show strong positive coefficients.
- This matches Day 1's Random Forest feature importance — cross-model agreement builds reviewer trust.

A.7 — Evaluating Logistic Regression





Same metric suite as Day 1: ROC curve (left) and confusion matrix (right) — enabling a true apples-to-apples comparison across every model we build.

B

THE BRIDGE

From One Neuron to a Neural Network

The big reveal: Logistic Regression IS a single artificial neuron.

B.1 — One Neuron IS Logistic Regression

1 Weighted Sum

$z = w \cdot x + b$ — identical to Logistic Regression's linear combination step.

2 Sigmoid Activation

$\sigma(z)$ — the exact same squashing function from Part A.

3 Training

Minimize cross-entropy loss via gradient descent — the exact same idea, automated by a deep learning framework.

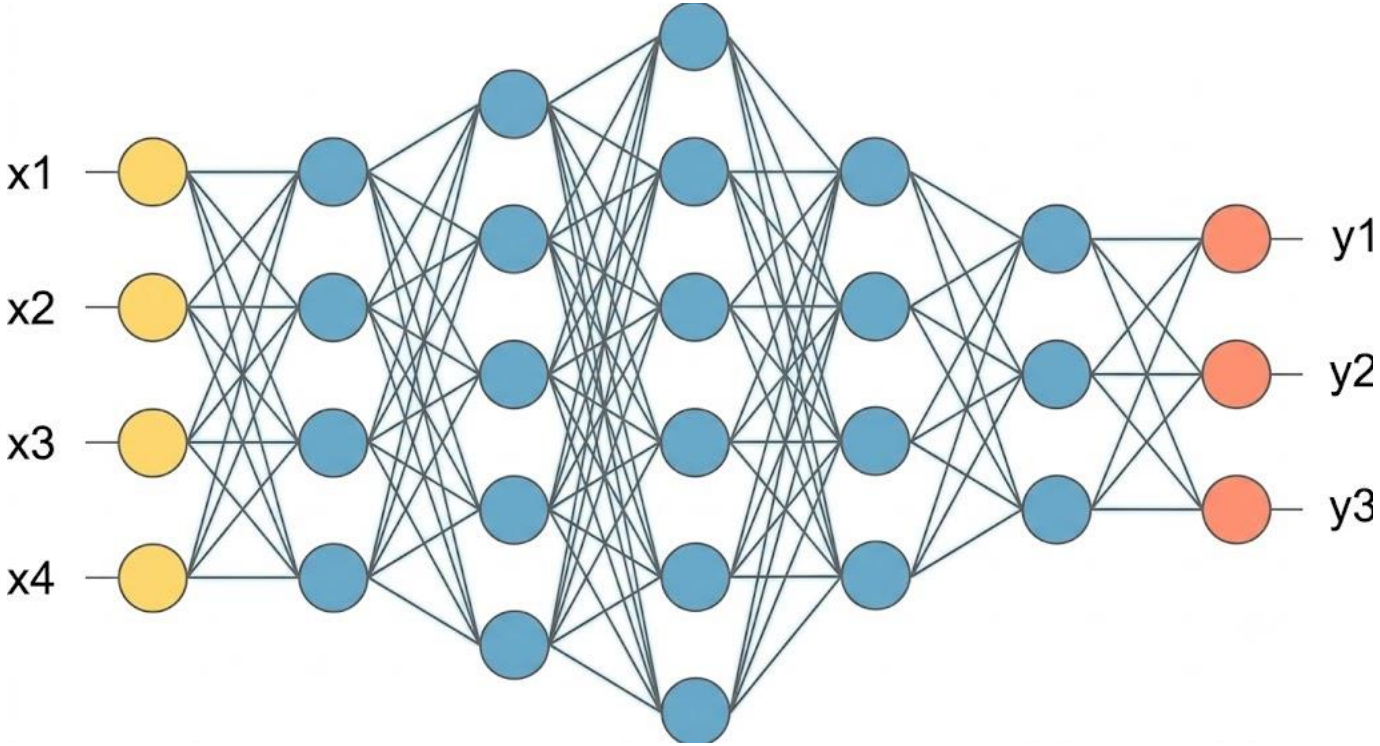
4 The Upgrade

Stack many neurons into layers, and many layers on top of each other, to learn non-linear feature combinations.

https://whyaxis.wordpress.com/2017/09/14/building-a-simple-neural-network-with-keras-and-tensorflow/

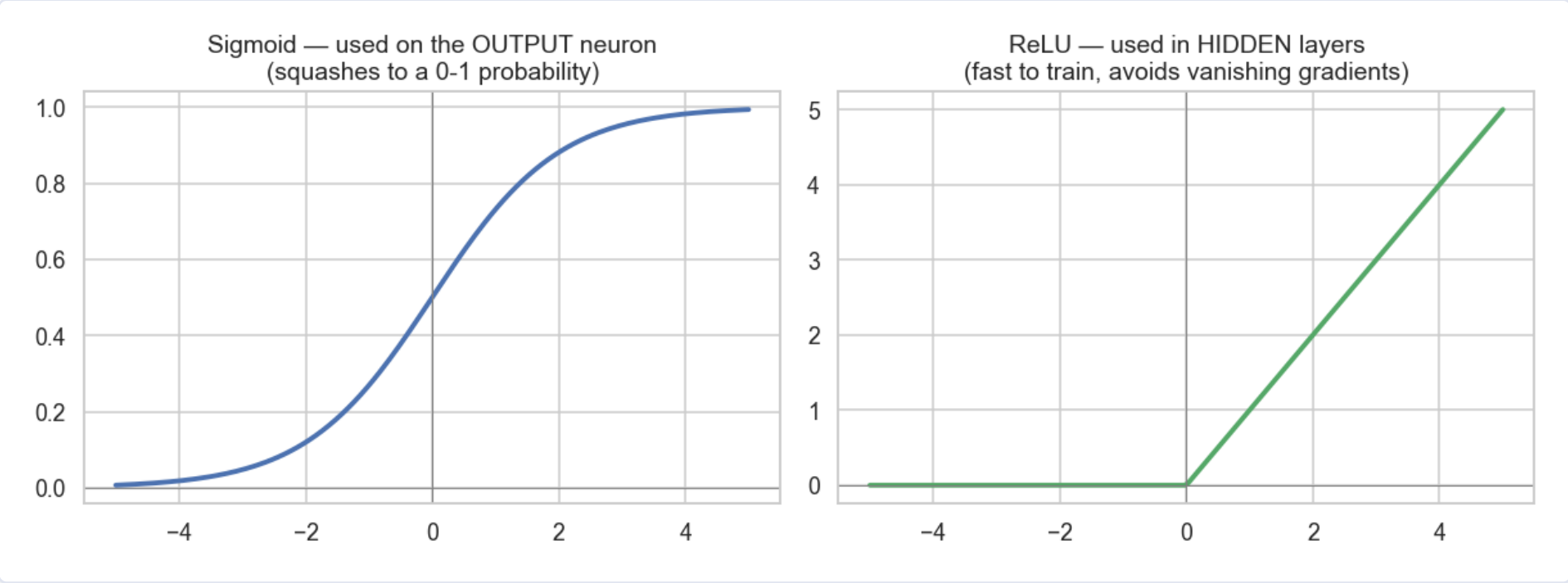
From Features to a Stroke Probability — Layer by Layer

Each hidden neuron learns its own "mini logistic regression" over the previous layer's outputs — stacked together, the network learns non-linear feature combinations a single Logistic Regression cannot express (e.g. "high glucose is only dangerous when combined with old age AND hypertension").



INPUT LAYER	HIDDEN LAYER 1	HIDDEN LAYER 2	OUTPUT NEURON
37 patient features	ReLU activation	ReLU activation	Sigmoid → P(stroke)

B.2 — Activation Functions: Sigmoid vs. ReLU



Sigmoid is kept on the final OUTPUT neuron (we need a 0–1 probability). ReLU(z) = max(0, z) is used in HIDDEN layers — it trains faster and avoids the vanishing-gradient problem.

B.3–B.4 — PyTorch Setup & Class Imbalance

```
import torch, torch.nn as nn

X_train_t = torch.tensor(X_train_scaled,
                         dtype=torch.float32)
y_train_t = torch.tensor(y_train.values,
    dtype=torch.float32).unsqueeze(1)

# Weight the rare positive class inside the loss
n_pos, n_neg = y_train.sum(), len(y_train)-y_train.sum()
pos_weight = torch.tensor([n_neg / n_pos])

criterion = nn.BCEWithLogitsLoss(
    pos_weight=pos_weight
)
```

Same idea as class_weight="balanced"

- BCEWithLogitsLoss combines sigmoid + cross-entropy in one numerically stable step.
- pos_weight up-weights the rare "stroke" class inside the loss function itself.
- Tensors are just numpy arrays that PyTorch can auto-differentiate through.

B.5 — Architecture 1: A Simple 2-Hidden-Layer MLP

```
class SimpleMLP(nn.Module):
    def __init__(self, n_features):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(n_features, 32),
            nn.ReLU(),
            nn.Linear(32, 16),
            nn.ReLU(),
            nn.Linear(16, 1),  # raw logit
        )

    def forward(self, x):
        return self.network(x)
```

Input → 32 → 16 → 1

- Input layer: our 37 engineered features.
- Two ReLU hidden layers progressively compress and combine features.
- Final layer outputs one raw logit — sigmoid is applied inside the loss for stability.

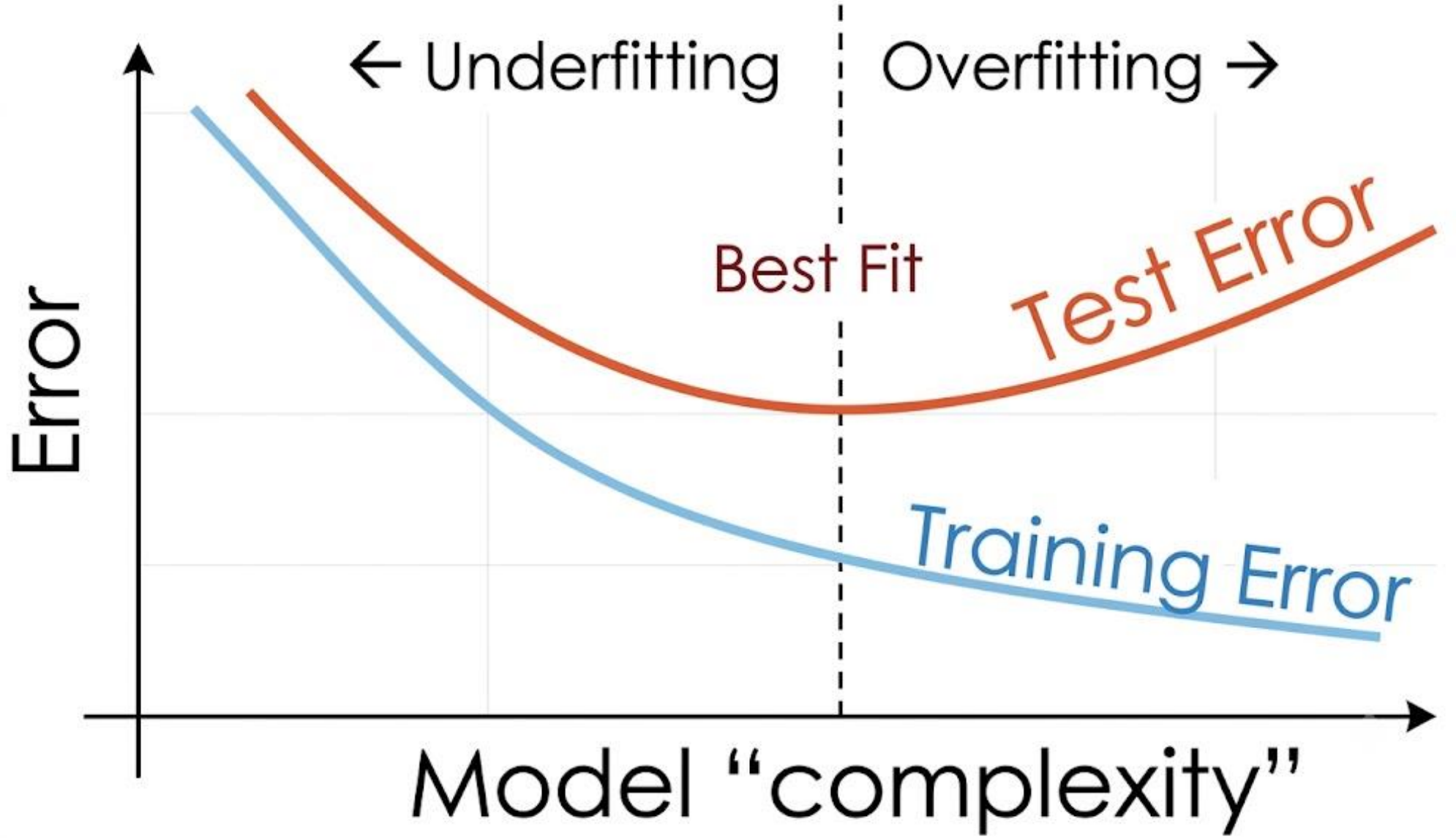
B.6 — Reading a Paper's Methods Section: Key Hyperparameters

Parameter	Controls	How to Choose It
epochs	Full passes over training data	Watch the loss curve — stop once it plateaus.
learning_rate (lr)	Step size of each update	Too high → diverges. Too low → painfully slow. 1e-3 is a solid Adam default.
batch_size	Samples per weight update	Smaller = noisier but sometimes better generalization. 32–128 typical.
optimizer	Weight-update algorithm	Adam (adaptive) is the modern default; SGD needs more tuning.
weight_decay	L2 regularization in optimizer	A small value (e.g. 1e-4) helps prevent overfitting, like C in Logistic Regression.

Reading a Training Curve

Simple MLP — Train vs. Test Loss

- Both curves should fall together early in training.
- If train_loss keeps falling but test_loss starts rising again — that's overfitting.
- Solutions: fewer epochs, more regularization, a simpler architecture, or more data.
- We'll watch for this exact pattern in every architecture that follows.


← Underfitting
Overfitting →
Best Fit
Test Error
Training Error
Error
Model "complexity"

B.7 — Architecture 2: Deeper MLP with Dropout & BatchNorm

```
nn.Sequential(
    nn.Linear(n_features, 64),
    nn.BatchNorm1d(64), nn.ReLU(),
    nn.Dropout(0.3),

    nn.Linear(64, 32),
    nn.BatchNorm1d(32), nn.ReLU(),
    nn.Dropout(0.3),

    nn.Linear(32, 16), nn.ReLU(),
    nn.Linear(16, 1),
)
```

Two powerful regularization tricks

- BatchNorm1d normalizes activations inside the network — stabilizes and speeds up training.
- Dropout(p) randomly "turns off" a fraction p of neurons each step — forces the network not to over-rely on any one neuron.
- Together, these are some of the most widely used tools to fight overfitting in deep learning.

B.8 — Architecture 3: 1D CNN — Automatic Feature Engineering

- Day 1: WE engineered features by hand (BMI category, age×hypertension, risk scores).
- A convolutional layer lets the network discover its own local feature combinations automatically.
- We treat the feature row as a 1D signal — a Conv1d filter slides across neighboring features, learning to combine them, the same spirit as our hand-made interaction features, but self-discovered.
- Caveat: our columns have no natural order (age next to bmi is arbitrary) — a great teaching tool, but tree models / MLPs are usually the stronger real-world choice for tabular clinical data.

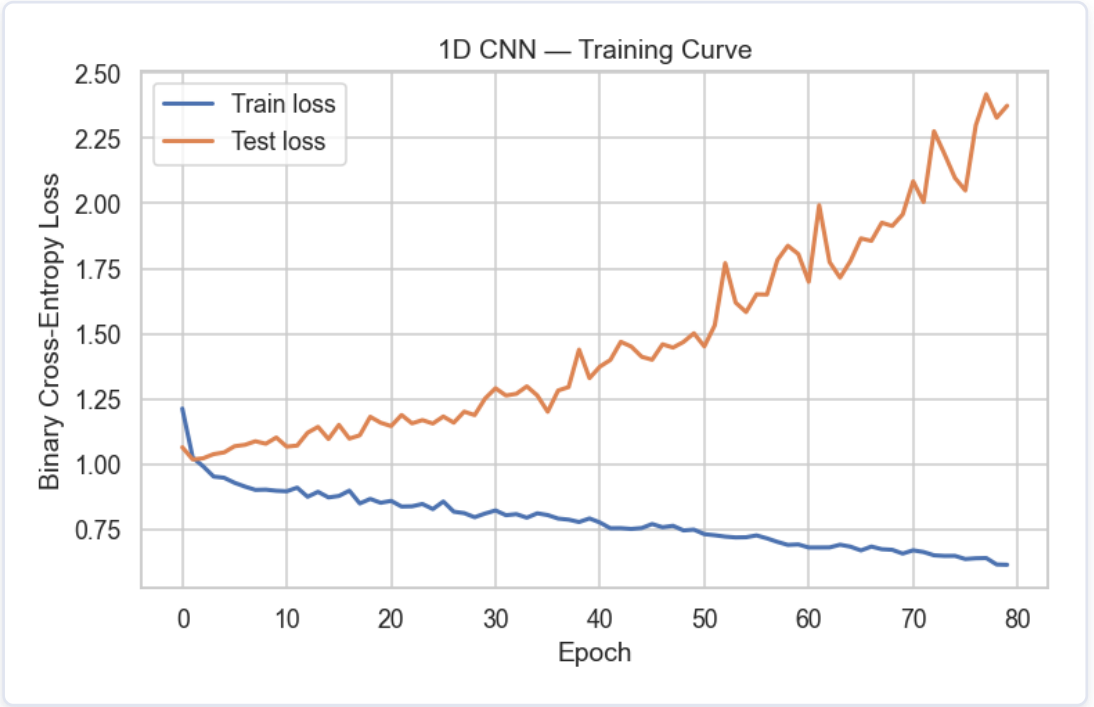
Conv1d & MaxPool1d — Parameter Guide

Parameter	Controls	Tuning Intuition
in_channels / out_channels	Filters in / learned filters out	More out_channels = more distinct patterns learned, more parameters.
kernel_size	Neighboring features per filter	Small (3) = local combos; tabular columns aren't ordered, so keep modest.
stride	Step between filter positions	1 = examine every position (most common default).
padding	Zero-padding at input edges	kernel_size // 2 keeps output length ≈ input length, no lost edges.
MaxPool1d(kernel_size)	Pooling window size	Downsamples, keeping the strongest signal. 2 is a common gentle default.

1D CNN — Training & Results

What did the CNN actually learn?

- Each filter learned its own small "recipe" combining neighboring input features — with no clinical knowledge given to it.
- Compare its Recall and ROC-AUC against the MLPs: on small tabular data, automatic feature learning rarely beats a good hand-engineered feature set.
- An honest, teachable result about WHEN convolution is worth its extra complexity — it truly shines on images, signals, and sequences.



Exercise — Build Your Own Neural Network

- Design and train a 3rd architecture, MyMLP, using the same nn.Sequential pattern.
- Try 1–3 hidden layers; reasonable neuron counts: 8, 16, 32, or 64 per layer.
- Use nn.ReLU() after each hidden nn.Linear; the final layer must output exactly 1 raw logit.
- Try adding nn.Dropout(0.2) after one hidden layer to observe its effect.
- Reuse the same train_pytorch_model(...) and get_pytorch_predictions(...) helpers — no need to rewrite the training loop!

Full worked example solution is provided in the companion Jupyter notebook.

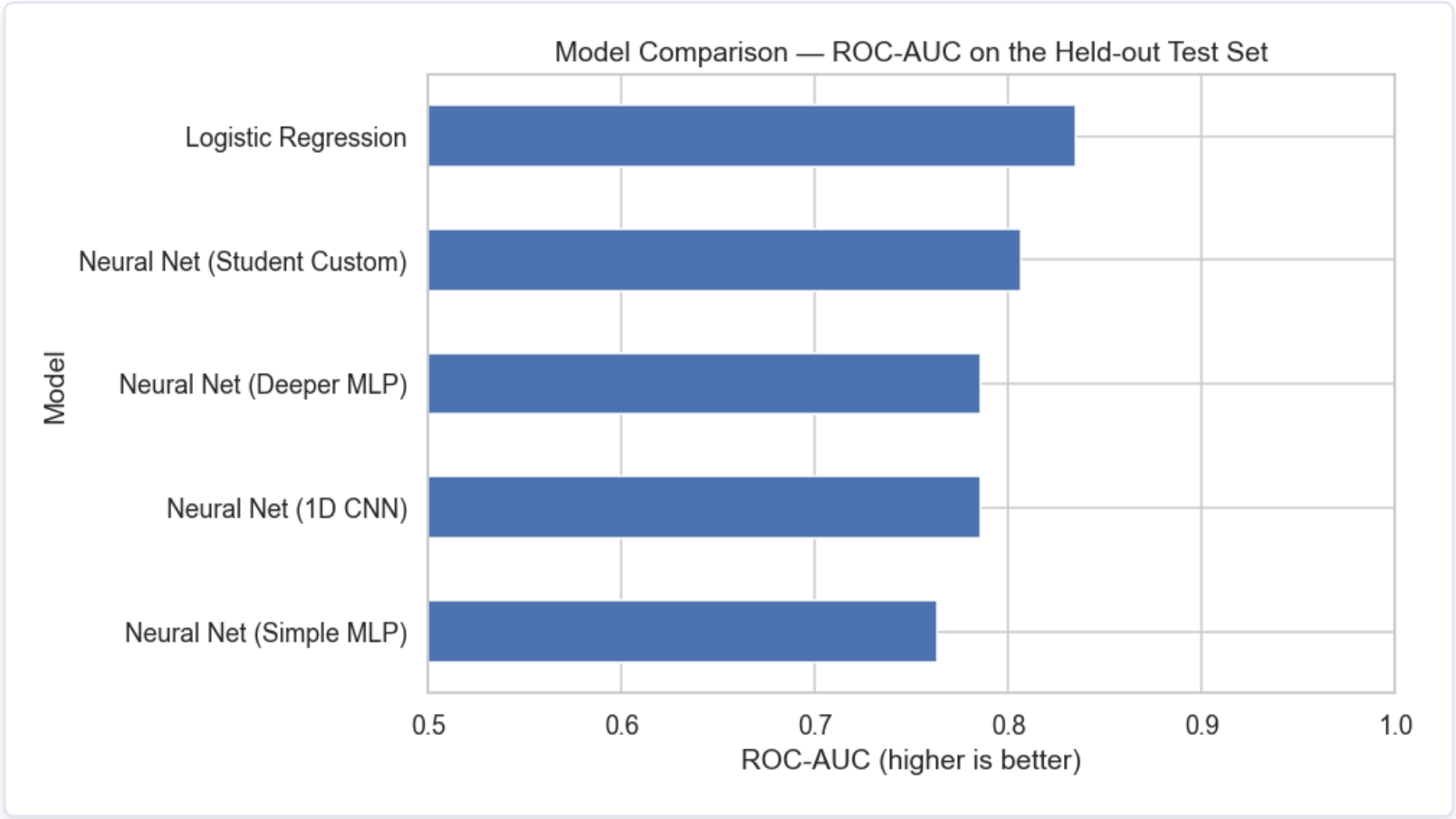
C

THE BENCHMARK

Comparing Every Model We Built

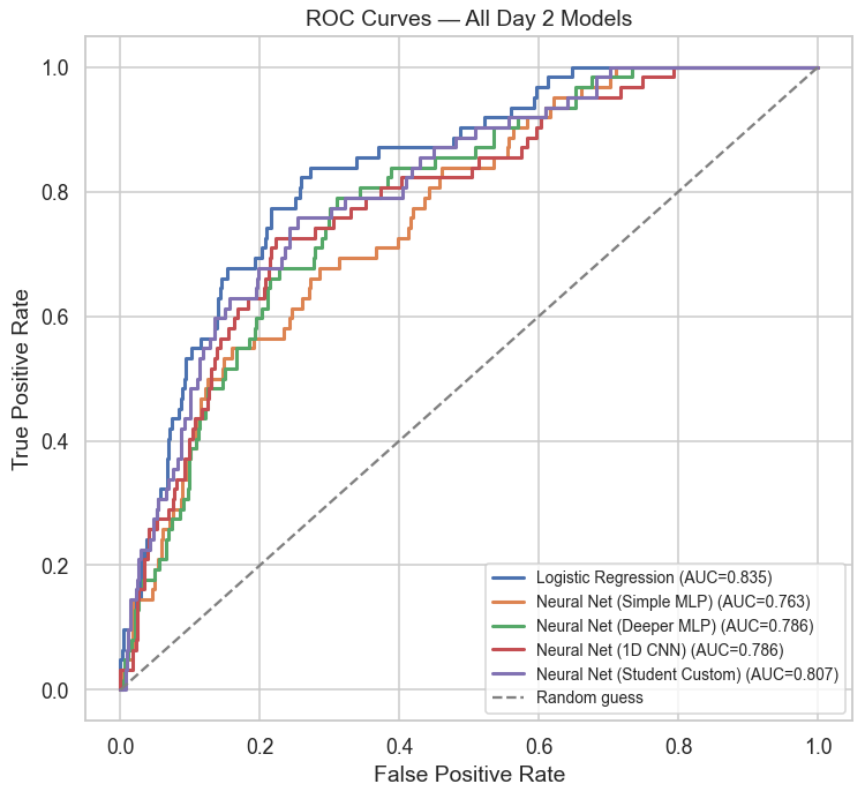
Six models, one held-out test set, one identical metric suite — the gold standard for a fair comparison.

ROC-AUC — All Models, Same Test Set



Ranked from lowest to highest ROC-AUC. A higher bar means better ranking quality across every possible decision threshold.

ROC Curves — All Day 2 Models



A very common, very important research finding

- On small-to-medium tabular clinical datasets (~5,000 rows, ~12 raw features), tree-based methods and even plain Logistic Regression often perform just as well — or better than — a deep neural network.
- Deep Learning tends to shine with a lot more data and/or unstructured inputs (images, text, signals).
- Knowing WHEN to reach for Deep Learning vs. classical ML is itself an important research skill — bigger and more complex is not always better.

D

WRAP-UP

Recap, Research Guidance & AI Ethics

From if/else statements to PyTorch neural networks — applied to a real, meaningful public-health problem.

D.1 — The Full Two-Day Journey

- ✓ Day 1: Python refresher — variables, operators, conditionals, loops, containers, functions, classes.
- ✓ Day 1: pandas EDA — missingness, class imbalance, rich exploratory analysis.
- ✓ Day 1: Feature engineering — imputation, clinical binning, encoding, composite risk scores.
- ✓ Day 1: Decision Tree → Random Forest → feature importance → full evaluation suite.
- ✓ Day 2: Logistic Regression from first principles — sigmoid, loss, gradient descent.
- ✓ Day 2: The bridge from one neuron to full neural networks.
- ✓ Day 2: PyTorch — Simple MLP, Deeper MLP, 1D CNN, and your own custom architecture.
- ✓ Day 2: Six models compared fairly on identical data with identical metrics.

D.2 — A Guide to Doing AI/ML Research the Right Way

- ✓ Define the clinical question first, then choose the model — not the other way around.
- ✓ Report your full pipeline: imputation, encoding, split ratio, and random seed.
- ✓ Always use a stratified split (or stratified k-fold CV) for imbalanced outcomes.
- ✓ Never report accuracy alone — always include Precision, Recall, F1, and ROC-AUC.
- ✓ Compare against a simple baseline (Logistic Regression) — complexity must earn its place.
- ✓ Interpret your model: feature importance, coefficients, or SHAP — build clinical trust.
- ✓ Be explicit about your dataset's limitations in your Discussion section.
- ✓ Make your code and (de-identified) data available wherever ethically and legally possible.

D.3 — Ethics & Responsible AI Use in Public Health

- Never upload real patient data to public, consumer AI/LLM tools without an approved, compliant data-processing agreement — even "anonymized" data can sometimes be re-identified.
- Perform real analysis locally or on your institution's approved, secured infrastructure.
- Always obtain proper ethics board (IRB) approval before working with real patient records.
- Be transparent in publications about which tools, models, and versions were used.
- A stroke-risk model is a decision-support tool, not a diagnostic replacement for a clinician — discuss potential biases in your training data (age, region, gender).
- Consider the downstream impact of false negatives vs. false positives, and choose your threshold accordingly.

Thank You!

You now have hands-on experience across the entire modern ML toolkit — from if/else statements all the way to PyTorch neural networks — applied to a real, meaningful public-health problem.

We hope this workshop becomes a genuine foundation for your thesis work and future research in AI for Public Health.

Teerapong Panboonyuen, Ph.D. (Kao) · College of Computing, Khon Kaen University

Acknowledgements: Kaggle (dataset) · scikit-learn · PyTorch · pandas